

Підготовка до захисту — «Нічний Дозор»

Документ містить відповіді на два блоки питань:

1. Які питання по коду може задати викладач
 2. Що таке константи, масиви та MonoBehaviour
-

Частина 1. Питання, які може задати викладач

1. Загальна архітектура

П: Зі скількох скриптів складається гра і за що кожен відповідає?

В: 11 файлів: TDGameBootstrap (запуск), GameManager (логіка), GameConfig (числа), UIManager (інтерфейс), TowerInput (миша), Tower, Enemy, Projectile, LevelMap, BuildZone, ModelSpawner.

П: Як гра запускається без ручного налаштування сцени?

В: У TDGameBootstrap метод з атрибутом [RuntimeInitializeOnLoadMethod] після завантаження сцени створює об'єкт NightWatchGame і додає GameManager, UIManager, TowerInput.

П: Де «мозок» гри?

В: GameManager — золото, хвили, будівництво, HP кристала, спавн ворогів, перемога/поразка.

П: Де зберігаються всі числа (урон, ціни, таймери)?

В: У GameConfig.cs — константи та методи розрахунку статів.

П: Що таке namespace NightWatch?

В: Спільний простір імен — усі класи гри в одному «пакеті», щоб не плуталися з чужим кодом.

2. Unity і життєвий цикл

П: Чим відрізняються Awake, Start, Update?

В:

- Awake — одразу при створенні об'єкта (singleton у GameManager)
- Start — один раз перед першим кадром (меню, прогрів моделей)
- Update — кожен кадр (таймер хвили, HUD, атака башен)

П: Що таке MonoBehaviour?

В: Базовий клас Unity-скрипта; дає Update, Start, доступ до transform, gameObject.

П: Що таке GameObject і Transform?

В: GameObject — об'єкт у сцені; Transform — позиція, поворот, масштаб.

П: Навіщо FindFirstObjectByType<UIManager>()?

В: Знайти вже створений компонент у сцені без жорсткого посилання в Inspector.

3. Патерни і структура

П: Що таке Singleton і де він використовується?

В: GameManager.Instance — один екземпляр на всю гру. У Awake: якщо другий — Destroy. Інші скрипти звертаються через GameManager.Instance.

П: Навіщо static class GameConfig?

В: Загальні налаштування без об'єкта в сцені — одне джерело балансу.

П: Що таке enum у вашому проєкті?

В: Списки варіантів: RaceType, TowerType, EnemyType, Difficulty, WaveRewardType.

П: Що таке struct (TowerStats, EnemyStats)?

В: Невеликий набір полів (HP, урон, швидкість) без окремого об'єкта в сцені.

4. GameManager (часто питають)

П: Як гравець починає гру після меню?

В: SelectRace() — скидання золота/хвили, BuildLevel() або ResetLevelState(), HP кристала, ShowGameHud().

П: Як ставиться башня?

В: TowerInput → клік → FindBuildZoneFromScreen → TryBuildAtZone → GameObject + Tower.Init().

П: Коли хвиля вважається пройденою?

B: У Update: WaveActive, _spawnDone == true, ActiveEnemies.Count == 0 → OnWaveComplete().

П: Як запускається хвиля?

B: Кнопка «Почати хвилю» → StartNextWave() → StartCoroutine(SpawnWave).

П: Де зберігається HP кристала?

B: У GameManager: CrystalHp, CrystalMaxHp; урон через DamageCrystal().

П: Що відбувається, коли час хвилі вийшов?

B: WaveOvertime = true, кристал отримує 2 HP/с, поки ворогів не знищать.

П: Коли показується нагорода після 4-ї хвилі?

B: У OnWaveComplete: якщо CurrentWave == 4 і нагороду ще не брали → OfferMilestoneReward() → пауза вибору (RewardChoicePending).

5. Корутини

П: Що таке корутина і навіщо вона?

B: Метод IEnumerator з yield return WaitForSeconds — спавн ворогів з паузами, не блокуючи гру.

П: Чому спавн не в Update?

B: Потрібні точні інтервали (1.15 с між ворогами, паузи міні-хвиль).

П: Що таке yield return?

B: «Зачекати N секунд і продовжити»; решта коду (башні, UI) працює далі.

6. LevelMap і BFS

П: Як вороги знають, куди йти?

B: LevelMap.BuildWorldWaypoints будує масив Vector3[]; ворог у Update йде від точки до точки (_wp).

П: Як будується шлях на сітці?

B: BFS у FindPath: черга, сусіди по 4 напрямках, тільки клітинки дороги.

П: Що таке BFS простими словами?

B: Пошук у ширину — хвилями від старту до кристала по дорозі.

П: Де можна будувати башні?

B: IsBuildable — не дорога, не дерево, не кристал, не край карти.

7. Tower, Enemy, Projectile

П: Як башня обирає ціль?

B: FindBestTarget — найближчий живий ворог у радіусі Combat.Range.

П: Чим відрізняються типи атак?

B: Single — один ворог; Aoe — мортира; Slow — заморозка; Chain — блискавка (до 3 цілей).

П: Як рахується урон з урахуванням раси і рівня?

B: GameConfig.GetTowerCombat() — база × раса × рівень × бонуси після 4-ї хвилі.

П: Що робить Projectile?

B: Летить до цілі; при попаданні — урон, АОЕ або slow.

П: Що особливого у боса?

B: 10-та хвиля, багато HP, періодично викликає Scout-міньонів.

8. UI (UIManager)

П: Чому UI створюється кодом?

B: Увесь інтерфейс збирається в BuildUi() — Canvas, панелі, кнопки.

П: Як UI дізнається золото і HP?

B: RefreshHud() кожен кадр читає GameManager.Instance.

П: Як не будувати башню при кліку по кнопці?

В: `UiEventSetup.IsPointerOverUi()` — якщо курсор над Canvas, клік ігнорується.

9. Механіки екзамєну (RANDOM.ORG)

Механіка	Де в коді
Стартовий вибір (складність + раса)	<code>BuildMenu</code> , <code>SelectRace</code> , <code>DifficultyConfig</code> , <code>RaceRate</code>
Часові обмеження	<code>GetWaveTimeLimit</code> , <code>TickWaveTimer</code> , <code>overtime</code>
Захист цілі	<code>CrystalHp</code> , <code>DamageCrystal</code> , шляхи <code>SpawnPaths</code>
Нагорода після 4-ї хвили	<code>WaveRewardConfig.PickRandomOffers</code> , <code>OfferMilestoneReward</code>

П: Звідки випадковість у нагородах?

В: `PickRandomOffers()` — перемішування масиву, беруться 3 перші.

10. Підступні питання

- Без `EventSystem` — кнопки UI не клікаються.
 - Ворог не йде напřаму — тільки по BFS-шляху по дорозі.
 - Продаж башні — 70% від `_totalSpent` (`SellRefundRate = 0.7f`).
 - Карта створюється в `BuildLevel()` при першому виборі раси.
-

Відповідь на 30 секунд (шаблон)

«Гра автозапускається через `TDGameBootstrap`. Логіка в `GameManager`: гравець обирає складність і расу, будує башні на `BuildZone`. Хвили запускає корутину `SpawnWave`, вороги йдуть по шляху з `LevelMap.FindPath`. Башні стріляють через `Projectile`. HP кристала в `GameManager`; якщо час хвили вийшов — `overtime` урон. Після 4-ї хвили — випадковий вибір з 9 нагород у `WaveRewardConfig`. Усі числа — в `GameConfig`.»

```
for (int i = 0; i < 6; i++)
    CreateButton(TowerNames[i]);
```

3. MonoBehaviour

MonoBehaviour — базовий клас Unity для скриптів на об'єктах у сцені.

```
public class GameManager : MonoBehaviour
public class Tower : MonoBehaviour
public class Enemy : MonoBehaviour
```

Без MonoBehaviour — звичайний C#-клас, Unity сам не викликає:

```
public static class GameConfig { ... }
public static class LevelMap { ... }
```

Що дає MonoBehaviour

Метод	Коли	У проєкті
Awake()	Об'єкт створено	Instance = this
Start()	Перед 1-м кадром	Меню, камера
Update()	Кожен кадр	Таймер, стрільба, рух ворогів

Доступ до Unity: transform, gameObject, Destroy(), AddComponent(), StartCoroutine().

Приклад

```
// GameManager – MonoBehaviour
void Update() {
    if (WaveActive) TickWaveTimer();
}

// GameConfig – MonoBehaviour
public static int KillGold(...) { ... }
```

Аналогія:

- MonoBehaviour — «живий» об'єкт у грі (башня, ворог, менеджер).
- static class — «довідник» з правилами і числами.

Все разом

```
// GameConfig – MonoBehaviour
public const int StartingGold = 120;
public static readonly string[] RaceNames =
{ "■■■■■", "■■■■■", "■■■■■" };

// GameManager – MonoBehaviour
public class GameManager : MonoBehaviour
{
    public int Gold = GameConfig.StartingGold;

    void Start() { _ui?.ShowMainMenu(); }

    public void SelectRace(RaceType race) {
        Gold = GameConfig.StartingGold;
        _ui?.SetMessage(GameConfig.RaceNames[(int)race]);
    }
}
```

Шпаргалка

Термін	Одна фраза
Константа	Фіксоване значення в GameConfig, не змінюється в грі
Масив	Список значень з індексами 0, 1, 2...
MonoBehaviour	Скрипт на об'єкті з Start/Update і доступом до сцени

Проект: «Нічний Дозор» — Tower Defense, Unity, namespace NightWatch.